Lidl Collect : Frontend

La web-app de click & collect, Lidl collect, permettant à un client de sélectionner un magasin, constituer un panier et suivre sa commande.

Périmètre de ce dépôt : frontend client et intégration PWA. Le backend (NestJS + PostgreSQL + Redis) est maintenu dans un dépôt séparé.

Démo : https://github.com/Alexandre74739/Lidl

STACK

COUCHE	TECHNOLOGIE
Framework UI	React + TypeScript
Build	Vite
PWA	vite-plugin-pwa (Workbox)
Styles	SCSS (architecture 7-1)
Routing	React Router
Lint	ESLint

React, SCSS et `vite-plugin-pwa` ont été sélectionnés après évaluation des alternatives disponibles sur npm.

ARCHITECTURE SOURCE

```
src/
├── assets/
│   ├── fonts/        # Police Antique Olive Nord D (charte Lidl)
│   └── images/
├── components/
│   ├── cart/         # CartEmpty, CartItemRow, CartSummary
│   ├── home/         # HeroSection, TerroirBanner
│   ├── layout/       # Header, Footer
│   ├── modal/        # GeolocalisationModal
│   ├── product/      # AvisCard, NutriScore, PourAccompagner, ProduitsSimilaires...
│   ├── rayons/       # AmbianceCard, AmbiancesGrid, PromoBanner, RayonCard...
│   └── ui/           # BottomNav, Button, DrawerMenu, InstallPrompt, ProductCard, Quantit
├── data/             # Données statiques (rayons, ambiances, menus)
├── pages/client/     # Home, Rayons, RayonDetail, ProductDetail,
│                     # Panier, Promotions, Fidelite, AmbianceMatch
├── services/         # CartContext, CookieContext, api.ts, productService, categoryServic
├── styles/
│   ├── abstracts/    # _variables.scss, _mixins.scss
│   ├── base/         # _reset.scss, _global.scss
│   ├── components/   # Un fichier par composant UI
│   └── layout/       # Un fichier par zone de mise en page
├── test-integ-back/  # Environnement de test d'intégration backend (hors production)
└── main.tsx
```

INTERFACES CLIENT

L'application expose huit vues : la page d'accueil, le catalogue par rayons, le détail d'un rayon, le détail produit, le panier, le compte utilisateur, la simulation d'achat. Chaque vue est un composant de page indépendant dans `src/pages/client/`, assemblé à partir de composants réutilisables issus de `src/components/`.

Le routing est déclaré dans `App.tsx` via React Router. `Header`, `Footer`, `BottomNav`, `GeolocalisationModal` et `InstallPrompt` sont montés en dehors des routes et persistent sur l'ensemble des vues.

La bibliothèque de composants est organisée en deux niveaux : les composants UI génériques (`Button`, `ProductCard`, `Quantity`) réutilisables sur l'ensemble de l'application, et les composants métier (`CartSummary`, `NutriScore`, `AvisSection`) propres à une fonctionnalité. Cette séparation permet de faire évoluer une interface sans impacter les autres.

GESTION DU PANIER

Le panier est géré via un Context React (`CartContext`) exposé à l'ensemble de l'application. Il centralise les articles, les quantités, le total et le calcul des économies réalisées à partir des prix barrés. Un composant consomme le panier via le hook `useCart` sans avoir à gérer l'état localement.

COUCHE RÉSEAU

La communication avec le backend repose sur `apiFetch`, un wrapper générique sur l'API Fetch native. Il injecte automatiquement le token JWT présent en session et normalise les erreurs HTTP. L'URL de base est lue depuis la variable d'environnement `VITE_API_URL`.

Deux services consomment cette couche : `productService` et `categoryService`, qui exposent les endpoints produits et catégories et typent leurs retours via les interfaces TypeScript `Product` et `Category`.

PWA

L'intégration PWA repose sur `vite-plugin-pwa`, sélectionné sur npm pour son intégration native dans le pipeline Vite. Il génère le Service Worker et le manifeste au build sans configuration supplémentaire.

Le manifeste configure l'application en mode `standalone` avec les icônes et couleurs de la charte Lidl, ce qui permet une installation depuis le navigateur sans passer par un store. Quatre stratégies de cache Workbox sont déclarées pour les fonts, les images et les appels API, avec des durées de rétention adaptées à chaque type de ressource.

Le composant `InstallPrompt` gère l'expérience d'installation sur deux environnements distincts. Sur Android et Chrome, il intercepte l'événement natif du navigateur et propose l'installation au clic. Sur iOS et Safari, cet événement n'existant pas, le composant affiche à la place la procédure manuelle. L'interface `BeforeInstallPromptEvent` a été définie manuellement, les types DOM TypeScript standard ne l'exposant pas. Le composant ne s'affiche pas si l'application est déjà installée.

DESIGN SYSTEM SCSS

L'ensemble des valeurs de charte (couleurs, typographie, espacements, breakpoints, ombres, z-index) est centralisé dans `_variables.scss`. Toute modification se propage à l'ensemble de l'application sans toucher aux composants.

Les tailles de police et les espacements utilisent `clamp()` pour adapter le rendu à la taille d'écran sans media queries sur la typographie. Les breakpoints sont définis en variables et consommés exclusivement via les mixins de `_mixins.scss`. Une variable `$nav-height` dédiée réserve l'espace de la navigation basse fixe en contexte PWA pour éviter que le contenu soit masqué.

GESTION DES COOKIES

Un second Context (`CookieContext`) wrape l'application et expose un bouton flottant persistant permettant à l'utilisateur de gérer ses préférences depuis n'importe quelle vue.

INSTALLATION

```
npm install
npm run dev                        # développement, Service Worker actif
npm run build && npm run preview   # production, port 4173
```

La variable `VITE_API_URL` doit pointer vers l'URL du backend. Sans elle, `apiFetch` utilise `http://localhost:3000/api` en fallback.